Linux Basics — The Absolute Minimum You Must Know

Several modules drop you into a **real Linux shell** inside a container. You do not need to be a Linux administrator to do these labs — you need about twenty commands and a handful of survival keys, all on this page. Read it once before your first lab, then keep it open beside the terminal.

> **Do you even need this yet?** Modules **05, 06, 09 and 10** run in your web browser — the console just opens the right page for you. Modules **01, 02, 03, 04, 07, 08 and 11** put you at a Linux prompt. That's where this page matters.
>
> **Two things this page deliberately does not cover, because they get their own lab:** writing **firewall rules** with `iptables` (module 07) and the **security tools** like `nmap` and John the Ripper (modules 02, 03). This page is the everyday shell those labs sit on top of — revise the relevant lab for the tool itself.

Reading the prompt

When a lab "logs you in", you'll see something like:

```
root@station:/office#
analyst@station:~$
```

That's **who** you are `@` **which machine** `:` **which directory you're standing in**, then `#` or `$`. A `#` means you are **root** (the all-powerful administrator); a `$` means a normal user. The `~` is shorthand for your **home directory**. Glance at the prompt whenever you're unsure — it answers "who am I and where am I?" before you ask.

Where am I? Moving around

```
pwd            # print the directory you're standing in
ls             # list the files here
ls -l          # long listing: permissions, owner, size
ls -a          # include hidden files (names starting with .)
cd /office     # go to a directory by absolute path
cd hashes      # go into a subdirectory (relative to here)
cd ..          # go up one level
cd             # jump back to your home directory
```

Two kinds of path: an **absolute** path starts with `/` and works from anywhere (`/var/log/wire.log`); a **relative** path is measured from where you're standing (`emails/phishing.eml`). Three abbreviations: `.` is the directory you're in, `..` is its parent, `~` is your home.

That also explains the launcher: `./start.sh` means "run the file `start.sh` *right here*". The shell won't run a program from the current directory by name alone — that's a safety feature — so you spell out the `./`.

`bash start.sh` does the same job without the `./` — and without needing the file to be executable, which is why the lab and exam sheets use that form. Both work.

Looking inside files

```
cat message.asc      # print the whole file at once — fine for short files
less /var/log/syslog # page through a long file
man nmap             # read a command's manual (opens in less)
sha256sum report.pdf # the file's unique fingerprint — a long "hash" that changes if one
byte changes
```

Inside `less` (and `man`): **space** or arrow keys to move, `/text` then Enter to search, `q` to quit. If your terminal ever seems frozen showing a wall of text with a `:` or `(END)` at the bottom, you're in a pager — press `q`.

The keys that save you

- **Tab** — autocompletes file and command names; press it twice to see the options. Use it constantly: it's faster than typing and it can't misspell `salaries.csv`.
- ↑ **(arrow-up)** — brings back previous commands to edit and re-run.
- **Ctrl-C** — stops the command that's currently running. **It does not copy.**
- **Ctrl-Shift-C** / **Ctrl-Shift-V** — copy and paste in the terminal. (Plain Ctrl-V does nothing here; Ctrl-C kills what you're running. This is the classic first-day trap.)
- **Open a terminal**: in the file manager, right-click in the folder and choose **Open Terminal Here**. That is how the exam sheet starts.
- `q` — leaves `less` and `man`.
- `exit` — leaves the current shell (or an `ssh` session).
- Trapped in `vi` / `vim` by accident? Press **Esc**, type `:q!`, press Enter.

Editing a file

```
nano answers.md
```

Type normally. **Ctrl-O** then Enter saves ("write **O**ut"); **Ctrl-X** exits. The shortcuts are printed along the bottom of the screen — `^` means Ctrl. That's the whole editor.

Everyday housekeeping

```
mkdir reports          # make a directory
cp secret.zip backup.zip   # copy
mv draft.txt final.txt     # move — also how you rename
rm secret.hash             # delete — there is no recycle bin
```

Permissions in one breath

```
$ ls -l salaries.csv
-rw-r----- 1 root hr 1204 Aug 15 10:02 salaries.csv
```

Read `rw-r-----` as three triplets — **owner**, **group**, **others** — each `r`ead, `w`rite, e`x`ecute. Here the owner (`root`) can read and write, the `hr` group can read, everyone else gets nothing. Change it numerically (`chmod 640 file` — 4=read, 2=write, 1=execute, added up per triplet) or symbolically (`chmod g+w file`). `chown root:hr file` sets owner and group; `id jane` shows which groups jane belongs to.

Module 01 builds this into real access control — this paragraph is just so the notation doesn't ambush you.

Combining commands

The shell's superpower is plumbing small commands together. Four fittings appear in the labs:

```
zip2john secret.zip > secret.hash      # >  sends output into a file (overwrites it)
ps aux | grep ssh                      # |  pipes output into the next command
cipher break "$(cat challenge.txt)"    # $( ) drops one command's output into another
wc -w texts/*.txt                      # *  matches every file ending in .txt
```

Quotes matter: `'single quotes'` keep text exactly as typed; `"double quotes"` still let `$(...)` and `$variables` expand. When a command's argument contains spaces or symbols, quote it.

`grep` deserves its own line — it finds text: `grep 'password=' wire.log` prints matching lines, `-c` counts them instead, `-a` forces a binary-ish file to be treated as text.

Being someone else, being somewhere else

```
sudo usermod -aG hr jane     # run one command as root
sudo -u rico cat report.txt  # run one command as a different user (rico)
```

```
ssh pc2                      # log into another machine — watch the prompt change
exit                         # ...and come back
ssh server 'wg show'         # run ONE command over there and return immediately
```

Note the difference: bare `ssh pc2` moves you there until you `exit`; `ssh host 'command'` comes straight back on its own — no `exit` needed.

What's running? What's listening?

```
ps aux               # every process; pipe into grep to find one
ps -ef               # the same thing, a different spelling — some guides use this one
ss -tlnp             # TCP ports something is listening on, and which process owns them
kill 1234            # end the process with id 1234
pkill -f NAME        # end every process whose command line contains NAME
```

These are how a defender asks a machine "what are you actually doing right now?" — you'll want them any time a lab makes you suspicious of a box. (`ps aux` and `ps -ef` list the same processes in slightly different columns; use whichever a worksheet shows.)

Check yourself: spot the right command

Every option below looks plausible. Several are commands AI assistants and Windows habits produce. Only one does the job. If you cannot say **why** the others are wrong, re-read the section named in brackets.

1. Which command shows the TCP ports something is listening on, and which process owns them? [investigating] a. `ps -tlnp` b. `ss -tlnp` c. `netstat -ports` d. `ip ports show`

2. A service runs as `na-adminpanel`. End every process whose command line contains that word. [stopping a service] a. `pkill -f na-adminpanel` b. `stop-service na-adminpanel` c. `pkill -u root` d. `killall --name na-adminpanel --confirm`

3. Produce the fingerprint of `export.csv` to compare with a published value. [hashing] a. `hash export.csv` b. `checksum /sha256 export.csv` c. `sha256sum export.csv` d. `md5 export.csv --algorithm 256`

4. Make a file readable by its owner and the group, and nothing for anyone else. [permissions] a. `chown 640 export.csv` b. `chmod 640 export.csv` c. `chmod ugo+r export.csv` d. `chmod 644 export.csv`

5. Read a long log file, one screen at a time. [reading files] a. `cat /srv/logs/auth.log` b. `tail /srv/logs/auth.log --pages` c. `less /srv/logs/auth.log` d. `open /srv/logs/auth.log --pager`

6. Your terminal says `command not found: netstat`. Why, and what do you use instead? [investigating]

Answers

1. **b.** `ps` lists processes, not ports; `netstat -ports` and `ip ports show` are not real options. See "What's running? What's listening?".
2. **a.** `stop-service` is PowerShell, not Linux. `-u root` filters by *user*, not name. `killall --name --confirm` is invented syntax. See "Who is doing what".
3. **c.** `hash`, `checksum` and `md5 --algorithm` are invented or the wrong algorithm. See "Looking inside files".
4. **b.** `chown` sets the *owner*, not the permission bits. `644` leaves the file world-readable. See the permissions section of module 01.
5. **c.** `cat` dumps it all at once, `tail --pages` is invented, `open` is a desktop action. See "Looking inside files".
6. `netstat` is legacy and often not installed. `ss -tlnp` is the modern equivalent. See "What's running? What's listening?".

Practice before (or between) labs

The best free practice ground is **OverTheWire: Bandit** — a security-themed wargame where each level is one small Linux puzzle, played over `ssh` from any terminal (levels 0–10 cover everything on this page). Each level's page lists the commands you might need, so it doubles as a guided tour.

Quick reference

Command	What it does
`pwd`	print current directory
`ls` / `ls -l` / `ls -a`	list files / with details / including hidden
`cd dir` / `cd ..` / `cd`	enter a directory / go up / go home
`cat file`	print a whole file
`less file`	page through a file (`q` quits, `/` searches)
`sha256sum file`	print the file's fingerprint (checksum / hash)

Command	What it does
`man cmd`	manual for a command (`q` quits)
`nano file`	edit a file (Ctrl-O save, Ctrl-X exit)
`mkdir` / `cp` / `mv` / `rm`	make dir / copy / move-rename / delete
`chmod 640 file`	change permissions (numeric or `g+w` style)
`chown user:group file`	change owner and group
`id user`	show a user's groups
`grep 'text' file`	find lines containing text (`-c` count, `-a` force text)
`cmd > file`	send output into a file
`cmd1 | cmd2`	pipe output into the next command
`"$(cmd)"`	use a command's output as an argument
`sudo cmd` / `sudo -u user cmd`	run as root / as another user
`ssh host` / `ssh host 'cmd'`	log in remotely / run one remote command
`ps aux` / `ps -ef`	list running processes (two spellings)
`ss -tlnp`	list listening ports
`exit`	leave this shell or ssh session
Tab / ↑ / **Ctrl-C**	autocomplete / previous command / stop